

Crossing the Layers and Dotting the Details: Systematic Exploration of Near-Memory Computing

Riselda Kodra¹, Rafael Medina², Marina Zapater³, Giovanni Ansaloni¹, David Atienza¹

¹Embedded Systems Laboratory (ESL), EPFL, Switzerland

²Integrated Systems Laboratory, ETH, Switzerland

³REDS Institute, University of Applied Sciences Western Switzerland (HES-SO), Switzerland

Email: {riselda.kodra, giovanni.ansaloni, david.atienza}@epfl.ch, medinar@iis.ee.ethz.ch, marina.zapater@heig-vd.ch

Abstract—Near-Memory Computing (NMC) addresses the Von Neumann bottleneck between memory and processing by bringing computation capabilities close to where data is stored. This approach opens a vast design space, which requires both a detailed view of the NMC processing units (PUs) architecture and a broad perspective of the entire system integrating them. To tackle this challenge, this paper presents a novel cross-layer methodology that relies on synthesizable hardware models of the NMC PUs, encapsulated as components for event-based full-system simulation. Our approach enables accurate assessment of the resource requirements and efficiency of near-memory units while exploring them *in context*, i.e., when integrated into a system executing complex workloads. Hence, the resulting framework facilitates the systematic design exploration of NMC solutions. To this end, it also includes a complete software stack that supports the implementation and benchmarking of NMC-accelerated applications. We illustrate its effectiveness by providing area, energy, and performance metrics for executing Machine Learning (ML) inference benchmarks across the host CPU and the PUs using different DRAM standards. Our cross-layer approach highlights the potential of NMC acceleration, demonstrating system-wide speedups of $17.4\times$ on average.

Index Terms—Near-memory computing, Full-system simulation, Cycle-accurate simulation, Machine Learning

I. INTRODUCTION

The computing demands of modern data-intensive applications have increased rapidly over the past decade. Although processing capabilities have, for the most part, kept up with these advancements, memory performance has lagged behind, creating an ever-increasing gap known as the *memory wall* [1]. In particular, memories often fail to provide sufficient bandwidth to processors and accelerators, stalling computation and causing latency and energy bottlenecks.

To address this limitation, Near-Memory Computing (NMC) strategies, which advocate placing computing logic near the memory structures [2], [3], offer a compelling solution. NMC reduces the need for data transfers between memory and processing elements, thus improving latency and energy efficiency. Moreover, the tight interface between NMC processing units (PUs) and memory enables both high bandwidth and parallelism. NMC capabilities can be enabled at various points in the memory hierarchy, resulting in varying levels of parallelism, energy savings, and integration complexity. Among these alternatives, bank-level NMC architectures, which interface PUs directly with DRAM banks, are particularly promising, as they offer the highest level of performance and energy savings

without requiring the redesign of highly optimized memory arrays [3].

The bank-level NMC paradigm opens a broad design space; indeed, recent solutions showcase widely different application domains, performance, and energy efficiency targets [2]–[8]. To navigate this space and design NMC architectures according to a set of requirements, simulation frameworks are key enablers. In this context, most proposed tools choose one of two perspectives. Works such as [9]–[12] focus on low-level simulation of the NMC hardware, allowing accurate estimations of the area, energy, and performance of the PUs. However, they neglect the impact of system integration, restricting the scope of exploration. Alternatively, other works [13], [14] adopt a full-system perspective, allowing for the assessment of the interplay among applications, OS, system hardware, and NMC DRAM. Nevertheless, these tools abstract from architectural implementations, offering limited insights on the cost/performance trade-offs that stem from different PU design choices. Hence, both strategies offer important, but partial assessments when exploring NMC solutions.

Addressing this gap, we introduce a novel cross-layer methodology that, as depicted in Fig. 1, bridges architectural exploration with full-system simulation to enable a holistic exploration of NMC designs. This approach employs cycle-accurate simulations of synthesizable hardware models to provide detailed area / timing / energy characterizations of the NMC architectures. At the same time, it considers their integration within entire computing systems, which are modeled at an event-based abstraction level. In addition, we provide a software stack that aids in the design and deployment of entire

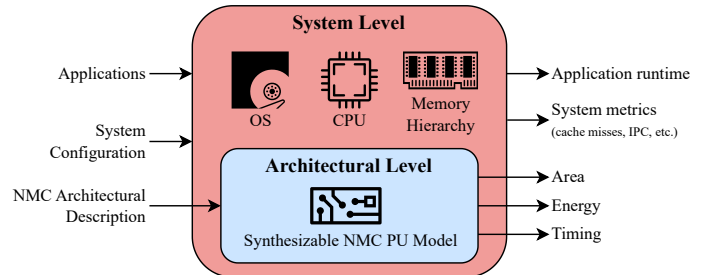


Fig. 1. Cross-layer simulation approach to evaluate NMC designs from both system and architectural perspectives.

machine learning applications, featuring the NMC acceleration of their computational kernels.

In summary, the contributions of this paper are as follows:

- We present a cross-layer simulation methodology that enables systematic exploration of bank-level NMC designs. The framework encompassing it integrates synthesizable hardware designs of NMC processing units within event-based simulations of entire computing systems.
- We introduce a dedicated software development stack that simplifies the development and execution of ML workloads on virtual NMC-accelerated systems.
- Demonstrating the capabilities of our approach, we assess runtime, area, and energy metrics of complete inferences across different ML benchmarks and DRAM protocols, accounting for the full system. Results show that NMC acceleration at the bank level effectively improves performance while employing a limited number of resources, with speed-ups exceeding $32\times$ for the case of SSDResNet34.

The introduced framework is available on GitHub¹ as open-source.

II. RELATED WORK

The bank-level Near-Memory Computing (NMC) solutions proposed in recent years [4]–[6] showcase a wide range of implementations, embodying different trade-offs between flexibility, efficiency, and performance. As examples, UPMEM [6] strives for flexibility by supporting a wide range of kernels through the implementation of complex PUs. Efficiency is, instead, the primary focus of domain-specific designs such as FIMDRAM [4] and Hynix-AiM [5], which achieve high performance and low energy by solely focusing on linear algebra kernels. Overall, the NMC paradigm offers high compute bandwidths, resulting in competitive performances with respect to GPU devices [18]–[20], with FIMDRAM claiming up to $2.1\times$ speed-up and 71% energy savings [21].

Similarly, NMC exploration strategies in the literature also take different approaches, primarily in terms of the abstraction level adopted, which is reflected in the precision of the results obtained and the required modeling effort, as summarized in Table I. Some works [5], [7] are based on prototyping and measurements of fabricated devices. This approach, while providing the most detailed characterizations, strictly limits architectural exploration due to high implementation costs, low flexibility, and slow design iterations. Other approaches [9]–[11], [16] involve functional-level simulators [22], DRAM simulators [23], [24] or a combination of them with custom-made profiling methods. In particular, MPUSim [9] and UniNDP [10] allow the configuration of architectural parameters to allow for flexible design exploration at a low implementation cost. However, their scope is constrained by a lack of assessment of their interaction within full systems.

To address this limitation, NDPmulator [14] and PIM-Sim [13] consider both the functionality of the design and the effect of the system parameters by employing system

simulators, such as gem5 [25]. NDPmulator [14] allows the user to define NMC architectures as gem5 memory objects that connect to the memory hierarchy. However, without low-level architectural definitions, allowed explorations cannot consider area and energy aspects. Moreover, its integration approach is inflexible, as it does not support interfaces that connect compute logic directly to DRAM hierarchical elements, such as in bank-level NMC. Meanwhile, PIMSim [13] can only instantiate a single design of the NMC architecture, preventing exploration opportunities. Additionally, it employs full-system simulation to evaluate the NMC design, rather than its integration into the system. Furthermore, although these works employ popular graph processing and linear algebra benchmarks to evaluate architectures, they do not provide any software support to develop new NMC-capable applications.

Bridging this gap, our methodology supports the exploration of bank-level NMC designs across abstraction levels from hardware to system, enabling a detailed evaluation of architectural choices and their impact on application-wide performance. It also encompasses a dedicated software stack to enable easy deployment of NMC-accelerated applications, focusing on CNNs. Consequently, it enables fast exploratory iterations by automatically linking architectural modifications with system-level assessments. The resulting strategy comprehensively and accurately assesses key metrics, allowing for holistic design exploration.

III. CROSS-LAYER EXPLORATION METHODOLOGY

The cross-layer methodology interfaces three main components, as depicted in Fig. 2:

- (a) A hardware design template for architecting NMC PUs, from which synthesizable instances with varying characteristics can be parametrically derived.
- (b) An event-based, full-system simulator interfacing the PUs as components in complete virtual computing systems, including models for processors, cache hierarchies, etc. The simulator supports behavioral implementations of memories for different DRAM standards.
- (c) A software stack aiding the definition of ML applications accelerated by NMC, relying on an Application Programming Interface (API) that handles their deployment on heterogeneous resources comprising CPUs and NMC PUs.

Integrated into the proposed framework, these components enable the holistic exploration of NMC solutions across the NMC PU architecture, system, and application abstraction layers. They are described in detail throughout this section.

A. NMC Architectural Template

Our framework comprises a NMC architectural template that enables the parametric definition of PU instances via design directives. Similarly to [4] and [12], the instances support Single-Instruction-Multiple-Data (SIMD) execution, governed by a minimalist instruction set comprising vector and vector-scalar operations such as addition, multiplication, multiply-add, and multiply-accumulate, as well as data movement instructions. The unconditional branch instruction allows for the execution

¹Open-source repository: <https://github.com/gem5-X/gem5-X-NMC>

TABLE I
COMPARISON OF NMC SIMULATION FRAMEWORKS

Framework	Arch. Exploration	Simulation Method	Software Support	Benchmarks	Metrics
PAISE [7]	No	Real device	C++ libraries	Transformer LLMs	Runtime, energy
uPIMulator [11]	No	Functional level	UPMEM Compiler [6]	Linear algebra, graph processing, NNs	Runtime
PIMsim [13]	No	Full-system (core only)	Standard C++	Graph processing	Runtime
MPUSim [9]	Yes (only PU [15])	Trace-based analysis	NVIDIA CUDA Compiler	Image Processing, Linear Algebra	Runtime, energy
MultiPIM [16]	No	Syscall emulation + functional level	C++ Pragmas	Graph processing [17]	Runtime
NDPmulator [14]	Yes (limited support)	Full-system	C++ libraries	Database applications, Linear Algebra	Runtime
UniNDP [10]	Yes	Cycle-accurate, instruction-driven	Compiler	MVM kernels (CNNs, LLMs)	Runtime
ANEMOS [12]	Yes	Cycle-accurate	Assembly templates	Linear Algebra	Runtime, energy, area
This Work	Yes	Full-System (cycle-accurate for NMC PUs)	C++ libraries	Entire ML models	Runtime, energy, area

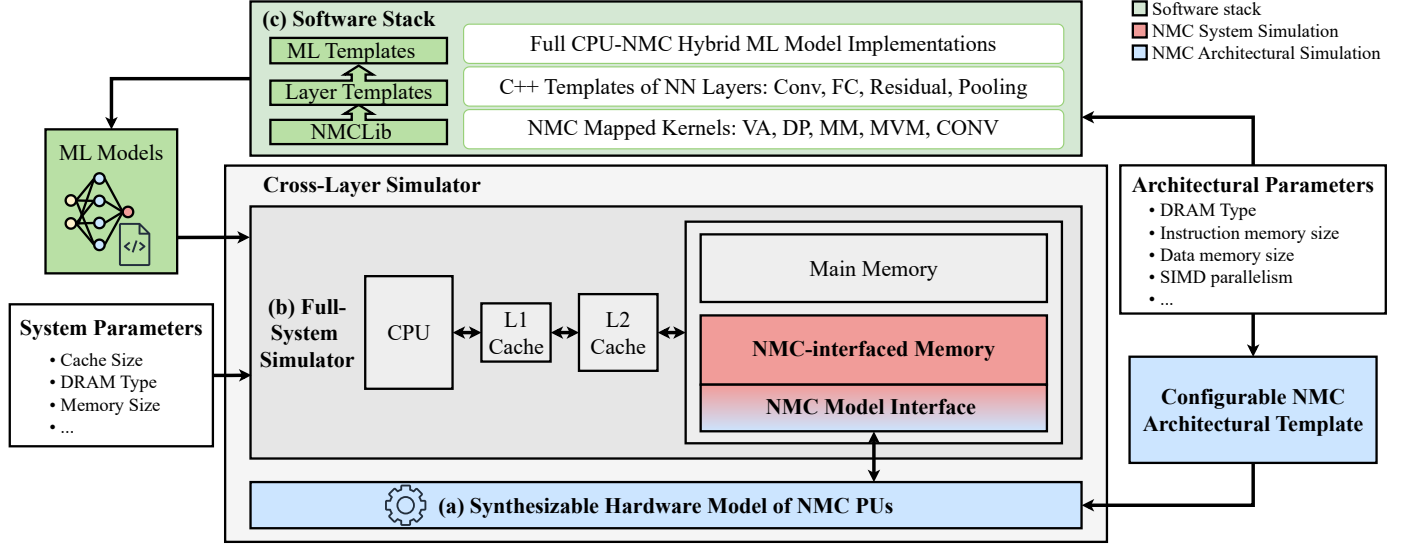


Fig. 2. Framework to assess the integration of NMC designs in a full system. It comprises (a) a synthesizable, cycle-accurate model of the NMC PUs, (b) a full-system simulator interfacing with the NMC model to evaluate NMC behavior according to the system and architectural parameters, and (c) a software stack allowing to offload kernels to the NMC PUs to accelerate the execution of ML workloads.

of loop nests. Instructions are stored in a Control Register File (CRF), while vector and scalar data are kept in the General and Scalar Register Files (GRF and SRF, respectively). As in [4], each processing unit is interfaced with two DRAM banks. The sizes of CRF, GRF and SRF are independent design parameters. Conversely, the SIMD width must adhere to the bank width of the considered DRAM standard.

Since the template is implemented in synthesizable SystemC, standard digital design flows can be employed to derive the area, energy, and timing characteristics of instances. Integration with full-system simulation is instead performed with the aid of a dedicated interface. Its aim is twofold: first, it propagates events to/from hardware and system (as discussed in Section III-B), so that a consistent state is maintained across them. Second, it connects PU instances to the modeled DRAM banks to enable their control via DRAM commands. Such an interface can be switched at run-time between two modes by writing to a memory-mapped control register. In memory mode, the host CPU accesses DRAM as a regular memory. When in NMC mode, all PUs are instead addressed in parallel by the DRAM commands. This mode enables the host to initialize the instruction and data buffers of all PUs simultaneously, and also to trigger the parallel execution of the stored instructions.

B. System Simulation Integrating NMC Models

The full-system simulator is responsible for modeling all the components of a target virtual system. To achieve this, our framework utilizes gem5-X [26], which is based on the popular gem5 environment [25] and follows an event-based approach. We interface it with the Ramulator DRAM simulator [23], which provides behavioral and timing models for the different DRAM standards. Building on these components, we herein extend the system simulator to provide an interface enabling the connection and abstraction of PU instances within the full system, as described in the previous section. From the system simulator perspective, the NMC-capable memory is instantiated as a gem5 system component, which extends the range of available main memory. The component wraps the DRAM simulator and the interface with the PU model. Simulations are then orchestrated across two threads, as illustrated in Fig. 3, with the first providing an event-accurate view of the overall system and the second providing a cycle-accurate view of the NMC PUs.

The NMC interface that connects the two threads monitors the accesses to the NMC memory range and guides the execution of the PUs accordingly. To avoid constantly alternating between threads and reducing the ensuing simulation overhead,

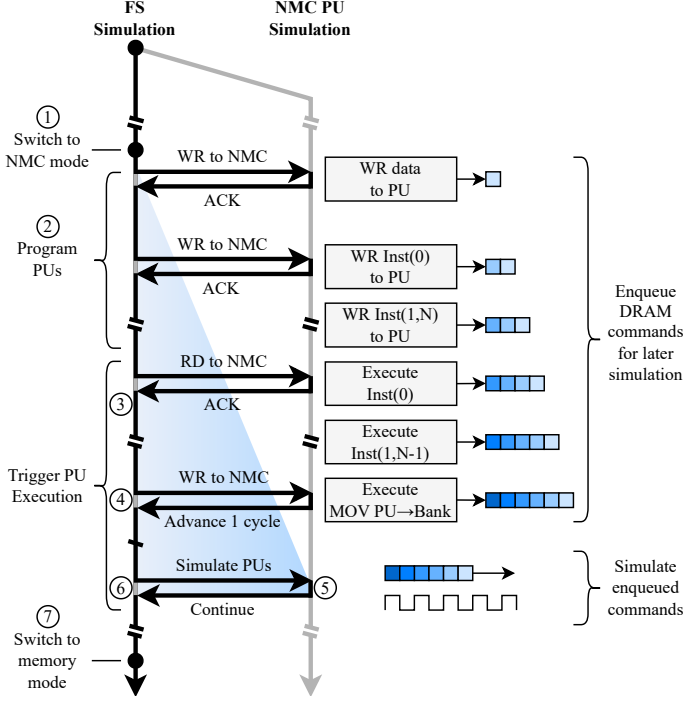


Fig. 3. Timing diagram of the execution of the cross-layer simulator, depicting the communication interface between the full-system and the models of the NMC PUs.

these accesses are enqueued if they do not change the data held in the DRAM banks (e.g., on read commands). When, instead, the bank data is modified (e.g., on writes triggering data movement from the PUs to the banks), the system simulation thread is paused, and the PU simulation thread processes the queue of commands. To accelerate the simulation time on the host, the cycle-accurate simulation is active only during this period. After all commands are consumed, the results are communicated to the NMC interface, which updates the state of the system simulator before resuming its execution. Both threads use a common clock, that seamlessly integrates event-based simulation with cycle-accurate one between the two threads.

In more detail, the execution of an NMC kernel proceeds as follows (see Fig. 3). First, the program issues a DRAM command to switch to the NMC mode ①. Next, a sequence of writes programs the data and instruction buffers of the PUs ②. These writes are captured by the interface and enqueued at the NMC model. Then, a sequence of reads triggers the execution of the PU instructions that do not modify the memory content, which are also captured and enqueued ③. When a write DRAM command prompts a write from the PUs to the memory banks ④, the command is enqueued, the system simulation pauses, and cycle accurate PU simulation becomes active ⑤. System simulation resumes once the PU models consume all their enqueued commands, sending the computation results to the NMC interface ⑥. This co-simulation pattern is repeated until the NMC kernel completes ⑦.

C. Software stack

Our framework includes a software stack (Fig. 2c) allowing users to easily develop and deploy NMC-accelerated machine learning workloads. Specifically, it targets the optimization of linear algebra kernels, the predominant computation patterns in machine learning algorithms. As detailed in the following, the software stack supports the description and execution of full ML networks (Fig. 4), relying on an API (described in Section III-C3) that defines the implementation and offloading of accelerated NN layers as linear algebra kernels to the NMC PUs.

1) *Definition of ML Models*: The software stack supports the high-level definition of models that contain both NMC-accelerated parts (e.g., convolution, fully connected, and residual layers) and CPU-hosted parts (such as pooling and normalization). The definition of models that run on such heterogeneous resources requires application programmers to instantiate the composing layers using templated definitions of neural network layers (extending TinyTensorLib [27]) as shown in the "Model Definition" block in Fig. 4, specifying their shape parameters and their connection with other layers in a header file. These definitions in turn employ optimized C++ arithmetic and data structures from Eigen [28].

2) *Implementation of NN Layers*: The software stack enables hardware-agnostic implementations of ML models by providing application programmers with methods to implement NN Layers for both CPU and NMC execution, as shown in the "Model Implementation" block in Fig. 4. For non-accelerated layers, the output of TinyTensorLib is directly employed for CPU execution. Conversely, layers earmarked for NMC acceleration are processed by a further library (NMCLib), which we developed and is described in the following paragraph. Supported layer types can be easily migrated from CPU to NMC PUs, facilitating the assessment of the benefits derived from near-memory execution.

3) *NMCLib API*: According to the DRAM and PU parameters, NMCLib handles the steps needed to accelerate linear algebra kernels with NMC. It automates the generation of both the DRAM commands required to program and execute the desired near-memory functionality and memory transfers to/from the compute memory address space, as shown in the "NMC Kernel Flow" block in Fig. 4. First, it allocates the DRAM memory space needed for kernel execution and lays out the data operands in memory for NMC execution, aligning the data structures with the bank organization. In particular, in order to maximize parallelism, NMCLib ensures that kernel inputs and outputs are tiled in consecutive addresses on the available compute memory banks. Next, the library generates the sequence of DRAM commands needed to program and execute the PUs. After the execution is complete, NMCLib manages the retrieval of results. The library supports vector addition (VA), dot product (DP), matrix-vector multiplication (MVM), matrix multiplication (MM), and convolution (CONV) kernels, generalizing the mappings proposed on [12]².

²Detailed technical documentation on the NMC Library is provided in the open-source framework repository: <https://github.com/gem5-X/gem5-X-NMC>

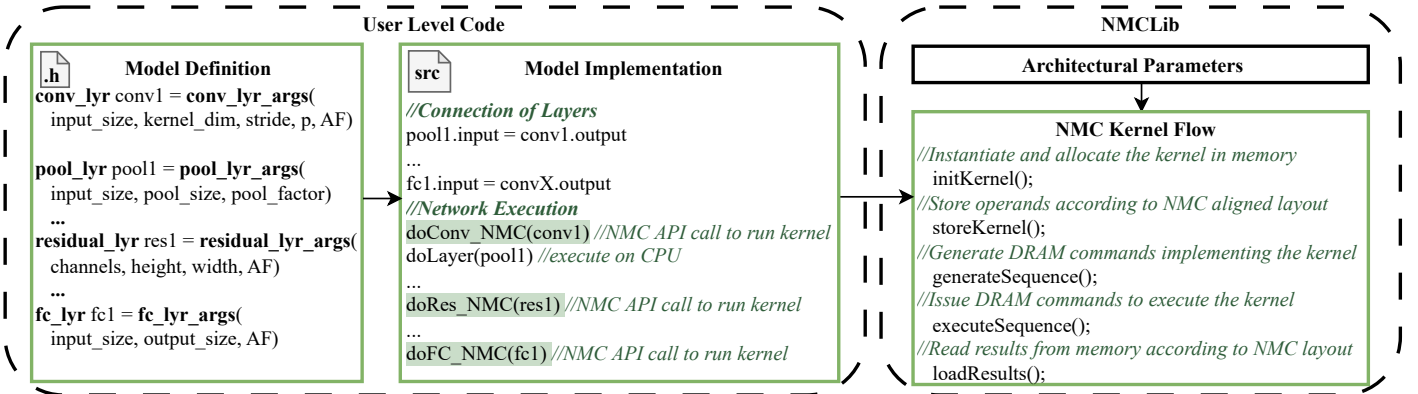


Fig. 4. Pseudocode illustrating the use of the software stack, enabling the definition of CNN models and their implementation utilizing NMC API calls to offload execution of layers to NMC PUs.

TABLE II
SYSTEM CONFIGURATION.

OS	Ubuntu 16.04
CPU	In-order ARMv8 core @ 2GHz with Neon SIMD Co-Processor [29]
L1 Instruction Cache	32 kB, 2-way, 2-cycle access
L1 Data Cache	32 kB, 2-way, 2-cycle access
L2 Cache	1 MB, 2-way, 20-cycle access
Main Memory	DDR4 2400 MHz 4 GB
NMC PUs	FP16, 32-instructions CRF, 8 registers per GRF and SRF

TABLE III
DRAM STANDARDS INTERFACED WITH NMC PUs.

	DDR4 [30]	GDDR5 [31]	HBM [32]	LPDDR4 [33]
Internal Clock (MHz)	400	1000	300	200
Number of Banks / PUs	16 / 8	16 / 8	16 / 8	8 / 4
Bank I/O Interface (bits)	64	256	256	256
PU SIMD Lanes	4	16	16	16

IV. EXPERIMENTS

A. Experimental Setup

We perform cross-layer explorations considering a host system comprising an ARMv8 CPU running Ubuntu Linux, and a multi-level cache hierarchy. NMC PUs are interfaced to a DRAM channel, using the architectural template described in Section III-A to model a FIMDRAM architecture [4]. Details of the system and PUs parameters are provided in Table II. As in FIMDRAM, each PU is connected to two banks. We integrate PUs into DRAM memory considering different standards, as shown in Table III. Besides operating at different frequencies, DRAM standards have a different number of banks per channel and I/O bitwidths at the bank interface, which determines the number of interfaced PUs and their SIMD width, as shown in the table. Area and energy results for NMC PUs are obtained using Catapult HLS and Cadence Genus, targeting a TSMC 28nm technology.

We considered application test benches performing inferences on three ML models: AlexNet [34], VGG16 [35] and SSDResnet34 [36]. Their characteristics are summarized in Table IV.

TABLE IV
ML BENCHMARKS.

Network	Number of Parameters	Number of layers			
		Conv.	FC	Pool.	Res.
AlexNet [34]	28.8 M	5	3	3	—
VGG16 [35]	15.3 M	13	2	5	—
SSDResNet34 [36]	28.2 M	53	1	2	16

B. Results

1) *Per-layer Performance Breakdown:* We first present a detailed view of the performance of the convolutional and fully connected layers of AlexNet by employing HBM DRAM. Fig. 5a shows the runtime of CPU and NMC-accelerated layers for both full-system (FS) and peak NMC performance implementations.

The FS NMC accelerated runtime is measured using our framework, accounting for the effects other system components have on the scheduling of DRAM commands. Instead, the NMC peak performance runtime is measured using a cycle-accurate simulation of the architectural template from Section III-A. In this simulation setup, only the NMC generation and execution phase are considered, and memory commands are issued at the maximum rate set by the considered DRAM protocol, following a similar approach as in [9]–[12]. Comparing the NMC FS and peak performance speed-ups, the latter results are highly optimistic with respect to the measurements in fabricated designs [4]. Instead, following our approach, we derive more realistic explorations, highlighting the importance of a system-aware stance. Results show different speed-ups (indicated as arrows) across network layers due to their differing dimensionality and access patterns. The largest kernels benefit the most from NMC acceleration (up to $49\times$ and $195\times$ for Conv2 FS and Peak Performance, respectively) since they can utilize the full parallelism of NMC PUs, present high data reuse, and have long enough execution times to offset configuration overhead.

More closely studying NMC performance from the full-system perspective, Fig. 5b illustrates the runtime breakdown of NMC execution phases for each layer. For convolutions, the generation and execution of the DRAM commands dominate the total runtime of the layers, confirming their viability as

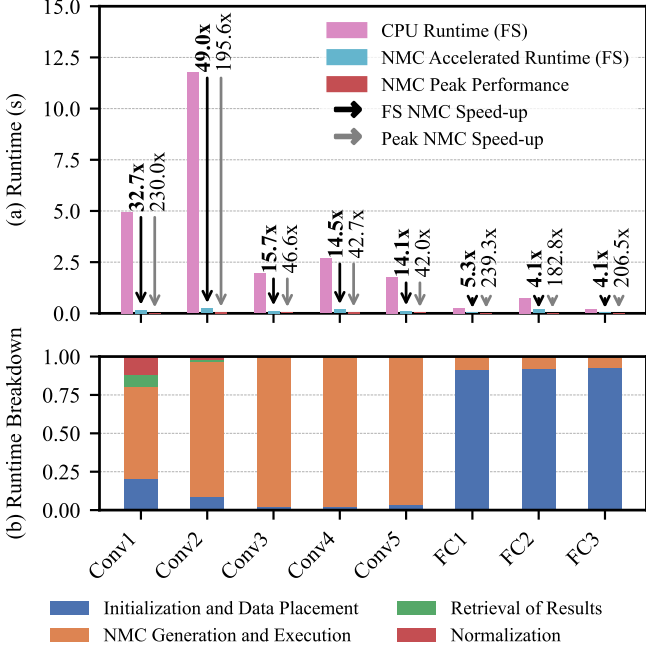


Fig. 5. (a) Runtime and speed-up of the AlexNet convolutional and fully connected layers when executed on a full system (FS) without NMC acceleration, on a FS with NMC acceleration, and on NMC PUs at their peak performance. (b) Runtime breakdown of the different execution phases on NMC-accelerated FS execution.

NMC acceleration targets. Instead, the initialization and read-back phases incur minimal runtime overhead. Lower speed-ups are observed for fully connected layers (up to $5\times$ for FC1), as their lower data reuse implies a larger data transfer cost.

2) *Full Network Performance*: In Fig. 6a we report the inference run-time of the benchmark ML models. Note that our methodology enables an accurate assessment of full applications, instead of modeling accelerated kernels in isolation, as in previous analyses [9]–[12]. The figure displays the runtime of NMC and CPU execution for each NMC DRAM standard, as well as the relative speed-up. The acceleration trends vary depending on the network and the type of DRAM. The networks affect the speed-up due to the dimensions of their layers, while interfacing PUs to different DRAM standards offers varying frequency and parallelism. The small size of the VGG16 layers limits the speed-up that can be obtained by integrating NMC PUs, while AlexNet and SSDResNet34 can attain accelerations of up to $26.0\times$ and $32.7\times$, respectively. Across all CNNs, GDDR5 offers the largest speed-ups thanks to its higher frequency and high parallelism. Instead, DDR4 and LPDDR4 showcase the lowest accelerations due to their lower parallelism.

Fig. 6b presents the runtime breakdown for the layer types in each network, both for NMC and CPU execution. The pooling layers are always executed on the CPU, while the other layer types can be implemented on either NMC or the CPU. In general, we observe that the layer types that can be accelerated on NMC dominate the overall CPU runtime, illustrating that the acceleration of linear algebra kernels is key in ML workloads.

TABLE V
COMPREHENSIVE PU RESULTS WHEN EXECUTING AN ALEXNET INFERENCE ON AN NMC-ACCELERATED SYSTEM.

DRAM Standard	Runtime (s)	Area (mm ²)	Energy (μJ)	Norm. EDP	Norm. EDAP
DDR4	1.88	0.10	8.29	1	1
GDDR5	0.95	0.42	10.05	0.61	2.61
HBM	1.12	0.32	12.37	0.92	2.83
LPDDR4	1.36	0.14	9.31	0.81	1.17

In particular, convolutional layers, which consistently take up most of the execution time, show the highest acceleration.

3) *Area and Energy Estimations*: The cross-layer approach also enables a detailed assessment of the cost (in terms of area and energy) of the different designs of NMC PUs. In fact, Fig. 7 shows the area occupation of the NMC PUs when integrated within a channel of different DRAM memories (left) and their energy consumption when running AlexNet (right). Area results show how the frequency and bank I/O width determine the PU size. For instance, although DDR4 operates at a higher frequency than HBM or LPDDR4, it achieves the smallest area due to its 64-bit datapath, as opposed to the 256-bit datapath of the other three standards. Energy results display smaller differences among the standards. Although DDR4 and LPDDR4 occupy a small area and thus have low static power consumption, their reduced parallelism implies longer runtimes, which increases dynamic consumption.

4) *Cross-layer Insights*: By providing the system and hardware metrics discussed above, our framework enables a holistic exploration of NMC solutions. Exemplifying this aspect, Table V displays the runtime, total PU area and energy, EDP (Energy-Delay Product), and EDAP (Energy-Area-Delay Product) when AlexNet is accelerated with NMC PUs interfaced to different DRAM standard channels. This analysis showcases the trade-off stemming from the costs of resources dedicated to NMC acceleration vis-a-vis the ensuing performance benefits. For example, although GDDR5 achieves the best performance and EDP, its large area footprint (due to a higher clock frequency) results in a higher EDAP than the slower DDR4 and LPDDR4 executions. Meanwhile, although HBM provides the best trade-offs in runtime and area, its static energy consumption deteriorates its EDAP compared to the other DRAM standards.

V. CONCLUSIONS

Near-Memory Computing (NMC) architectures promise to alleviate the memory bottlenecks experienced in current systems. Interfacing the processing logic with the memory structures enables highly parallel, low-energy computing. Although multiple solutions have been proposed in recent years, the exploration of this space is hampered by the lack of holistic methodologies. Architectural tools focus on hardware metrics, ignoring the behavior of the NMC processing units (PUs) in context, i.e., when integrated into complete systems. In contrast, frameworks for simulating NMC from a system-wide perspective often neglect architectural considerations, which

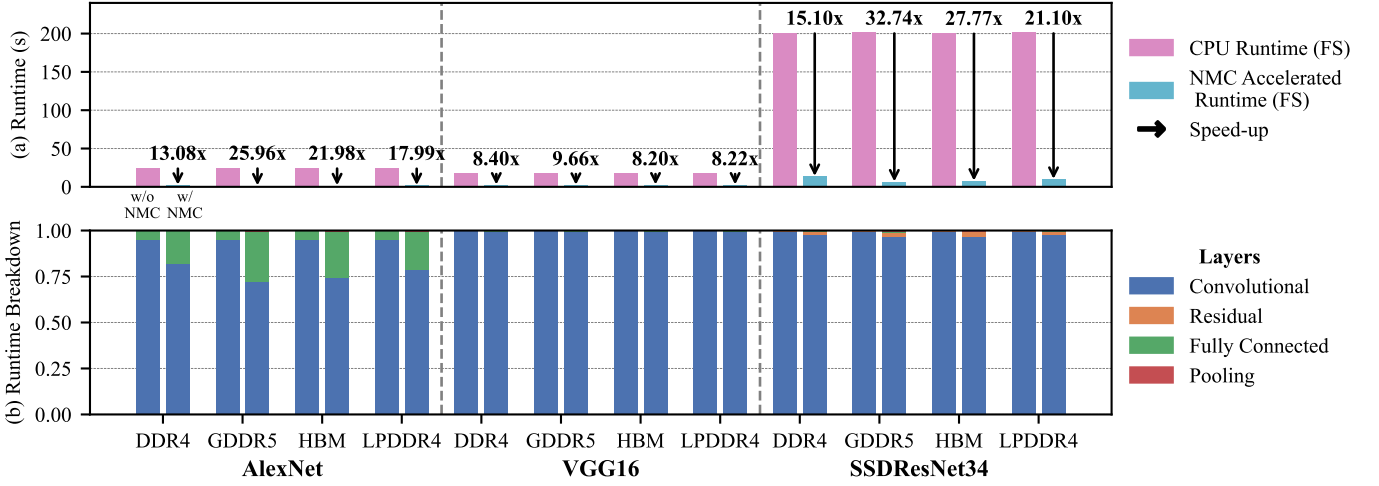


Fig. 6. (a) Runtime and speed-up of the considered CNN workloads when executed on a full system (FS) without NMC acceleration and on an FS with NMC PUs interfaced to different DRAM memories, and (b) runtime breakdown of the execution of the different layers composing the network.

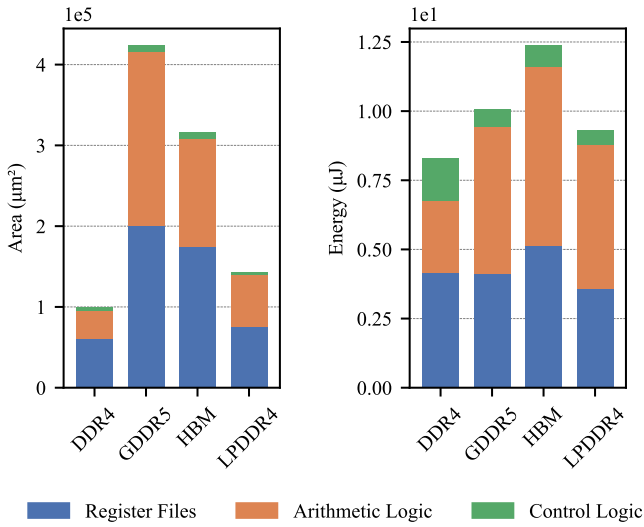


Fig. 7. Left: per-channel area occupation of the NMC PUs. Right: energy consumption of the PUs when executing AlexNet inference across DRAM standards.

limits their use in evaluating hardware design choices. To bridge this gap, in this paper, we have presented a new cross-layer strategy that interfaces full-system simulation with accurate architectural models of the NMC PUs. Consequently, we enable a detailed assessment of the PUs, allowing for high configurability and accurate estimation of their resource usage while also taking into account their interaction with other hardware/software system components. In addition, to facilitate the rapid benchmarking of solutions, we also provide a software stack that supports the implementation of ML applications utilizing NMC-accelerated kernels. Demonstrating the capability of our approach, we employed it to evaluate the behavior of a system integrating a modeled state-of-the-art FIMDRAM NMC design. Our framework showcases the

potential of NMC from a full-system perspective, achieving an average $17.4\times$ speed-up across the evaluated DRAM standards and ML models, even when the interplay between the CPU host, the memory hierarchy, and the acceleration of NMC is taken into account.

ACKNOWLEDGMENTS

This work has been partially supported by a joint research grant for ESL-EPFL by IMEC, by the Swiss NSF Edge-Companions project (GA No. 10002812), by the EC HORIZON CERBERUS project (GA No. 101223271) and by the Swiss State Secretariat for Education, Research, and Innovation (SERI) through the SwissChips research project. This research was partially conducted by ACCESS—AI Chip Center for Emerging Smart Systems, supported by the InnoHK initiative of the Innovation and Technology Commission of the Hong Kong Special Administrative Region Government.

REFERENCES

- [1] W. A. Wulf and S. A. McKee, "Hitting the memory wall: implications of the obvious," *SIGARCH Comput. Archit. News*, vol. 23, no. 1, p. 20–24, Mar. 1995. [Online]. Available: <https://doi.org/10.1145/216585.216588>
- [2] G. Singh, L. Chelini, S. Corda, A. Javed Awan, S. Stuijk, R. Jordans, H. Corporaal, and A.-J. Boonstra, "A review of near-memory computing architectures: Opportunities and challenges," in *2018 21st Euromicro Conference on Digital System Design (DSD)*, 2018, pp. 608–617.
- [3] C. Sudarshan, M. H. Sadi, L. Steiner, C. Weis, and N. Wehn, "A critical assessment of dram-pim architectures - trends, challenges and solutions," in *Embedded Computer Systems: Architectures, Modeling, and Simulation: 22nd International Conference, SAMOS 2022, Samos, Greece, July 3–7, 2022, Proceedings*. Springer-Verlag, 2022, p. 362–379.
- [4] S. Lee, S. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. Kim, "Hardware architecture and software stack for pim based on commercial dram technology: Industrial product," in *Proceedings - 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture, ISCA 2021*. Institute of Electrical and Electronics Engineers Inc., Jun. 2021, pp. 43–56.

- [5] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimirov, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Jang, I. Park, J. Chun, and J. Cho, "A 1ynm 1.25v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.
- [6] F. Devaux, "The true processing in memory accelerator," in *2019 IEEE Hot Chips 31 Symposium (HCS)*, 2019, pp. 1–24.
- [7] H. Lee, D. Baek, J. Son, J. Choi, K. Moon, and M. Jang, "Paise: Pim-accelerated inference scheduling engine for transformer-based llm," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1707–1719.
- [8] X. Xie, P. Gu, Y. Ding, D. Niu, H. Zheng, and Y. Xie, "Mpu: Towards bandwidth-abundant simt processor via near-bank computing," 2021. [Online]. Available: <https://arxiv.org/abs/2103.06653>
- [9] X. Xie, P. Gu, J. Huang, Y. Ding, and Y. Xie, "Mpu-sim: A simulator for in-dram near-bank processing architectures," *IEEE Computer Architecture Letters*, vol. 21, no. 1, pp. 1–4, 2022.
- [10] T. Xie, Z. Zhu, B. Li, Y. He, C. Li, G. Sun, H. Yang, Y. Xie, and Y. Wang, "UniNDP: A Unified Compilation and Simulation Tool for Near DRAM Processing Architectures," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. Los Alamitos, CA, USA: IEEE Computer Society, 2025, pp. 624–640. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/HPCA61900.2025.00054>
- [11] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding future pim architectures by demystifying a commercial pim technology," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 263–279.
- [12] R. Medina, G. Ansaloni, M. Zapater, A. Levisse, S. Alinezhad Chamazcoti, T. Evenblij, D. Biswas, F. Cathoor, and D. Atienza, "Bank on compute-near-memory: Design space exploration of processing-near-bank architectures," *Trans. Comp.-Aided Des. Integ. Cir. Sys.*, vol. 43, no. 11, p. 4117–4129, Nov. 2024. [Online]. Available: <https://doi.org/10.1109/TCAD.2024.3442989>
- [13] S. Xu, X. Chen, Y. Wang, Y. Han, X. Qian, and X. Li, "Pimsim: A flexible and detailed processing-in-memory simulator," *IEEE Computer Architecture Letters*, vol. 18, no. 1, pp. 6–9, 2019.
- [14] J. Vieira, N. Roma, G. Falcao, and P. Tomás, "Ndpmlator: Enabling full-system simulation for near-data accelerators from caches to dram," *IEEE Access*, vol. 12, pp. 10349–10365, 2024.
- [15] X. Xie, P. Gu, Y. Ding, D. Niu, H. Zheng, and Y. Xie, "Mpu: Towards bandwidth-abundant simt processor via near-bank computing," 2021. [Online]. Available: <https://arxiv.org/abs/2103.06653>
- [16] C. Yu, S. Liu, and S. Khan, "Multipim: A detailed and configurable multi-stack processing-in-memory simulator," *IEEE Computer Architecture Letters*, vol. 20, no. 1, pp. 54–57, 2021.
- [17] S. Beamer, K. Asanović, and D. Patterson, "The gap benchmark suite," 2017. [Online]. Available: <https://arxiv.org/abs/1508.03619>
- [18] C. Li, Z. Zhou, Y. Wang, F. Yang, T. Cao, M. Yang, Y. Liang, and G. Sun, "Pim-dl: Expanding the applicability of commodity dram-pims for deep learning via algorithm-system co-optimization," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 879–896. [Online]. Available: <https://doi.org/10.1145/3620665.3640376>
- [19] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, *PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference*. New York, NY, USA: Association for Computing Machinery, 2025, p. 862–881. [Online]. Available: <https://doi.org/10.1145/3676641.3716267>
- [20] J. Gómez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, pp. 52565–52608, 2022.
- [21] Y.-C. Kwon, S. H. Lee, J. Lee, S.-H. Kwon, J. M. Ryu, J.-P. Son, O. Seongil, H.-S. Yu, H. Lee, S. Y. Kim, Y. Cho, J. G. Kim, J. Choi, H.-S. Shin, J. Kim, B. Phuah, H. Kim, M. J. Song, A. Choi, D. Kim, S. Kim, E.-B. Kim, D. Wang, S. Kang, Y. Ro, S. Seo, J. Song, J. Youn, K. Sohn, and N. S. Kim, "25.4 a 20nm 6gb function-in-memory dram, based on hbm2 with a 1.2tflops programmable computing unit using bank-level parallelism, for machine learning applications," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64, 2021, pp. 350–352.
- [22] D. Sanchez and C. Kozyrakis, "Zsim: fast and accurate microarchitectural simulation of thousand-core systems," in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, ser. ISCA '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 475–486. [Online]. Available: <https://doi.org/10.1145/2485922.2485963>
- [23] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer Architecture Letters*, vol. 15, pp. 1–1, 03 2015.
- [24] P. Rosenfeld, E. Cooper-Balis, and B. Jacob, "Dramsim2: A cycle accurate memory system simulator," *IEEE Computer Architecture Letters*, vol. 10, no. 1, pp. 16–19, 2011.
- [25] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," vol. 39, no. 2, p. 1–7, Aug. 2011. [Online]. Available: <https://doi.org/10.1145/2024716.2024718>
- [26] Y. M. Qureshi, W. Simon, M. Zapater, K. Olcoz, and D. Atienza, "Gem5-x: A many-core heterogeneous simulation platform for architectural exploration and optimization," *ACM Transactions on Architecture and Code Optimization*, vol. 18, pp. 1–27, 07 2021.
- [27] J. Klein, I. Boybat, G. Ansaloni, M. Zapater, and D. Atienza, "Which coupled is best coupled? an exploration of aimc tile interfaces and load balancing for cnns," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 10, pp. 1780–1795, 2024.
- [28] G. Guennebaud, B. Jacob *et al.*, "Eigen v3," <http://eigen.tuxfamily.org>, 2010.
- [29] ARM. (2009) Introducing neon.
- [30] JEDEC, "Ddr4 sdram standard," JESD79-4D, 2021.
- [31] —, "Graphics double data rate (gddr5) sgram standard," JESD212C.01, 2023.
- [32] —, "High bandwidth memory (hbm) dram," JESD235D, 2021.
- [33] —, "Low power double data rate 4 (lpddr4)," JESD209-4D, 2021.
- [34] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, p. 84–90, May 2017. [Online]. Available: <https://doi.org/10.1145/3065386>
- [35] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [36] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "Mlperf inference benchmark," in *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, ser. ISCA '20. IEEE Press, 2020, p. 446–459. [Online]. Available: <https://doi.org/10.1109/ISCA45697.2020.00045>